Universidad Nacional Autónoma de México

Computer Engineering

COMPILERS

Parser and SDT

TEAM 06 MEMBERS:

423054589 - Arroyo Onofre Leonardo

319061345 - Camacho Ignacio Violeta

320047822 - Castañeda Luna Valeria

423020441 - Martínez Pérez Diego Alexander

320263635 - Ruiz Ramírez Santiago

320284858 - Saucedo Serralde Alejandra Melissa

GROUP: #5
SEMESTER:
2026-II

México, CDMX. April 2026

Contents

1 Introduction **2**

2 Theoretical Framework **3**
2.1 Compilers and Parsing 3
2.2 LL(1) Parsing 3
2.3 FIRST and FOLLOW Sets 4
2.4 Syntax-Directed Translation (SDT) 5
2.5 Abstract Syntax Tree (AST) 6

3 Development **6**
3.1 System Pipeline 6
3.2 Project Files 7
3.3 Enriched Lexer 7
3.4 Extended Grammar for Arithmetic Expressions 8
3.5 LL(1) Parser Implementation 8
3.6 Semantic Rules (SDT) 9

4 Results **9**
4.1 Test 1: Valid Declaration 9
4.2 Test 2: Valid Arithmetic Expression 12
4.3 Test 3: Semantic Error 13
4.4 Test 4: Syntax Error 15
4.5 Generated Output Files 15

5 Conclusions **16**

6 References **16**

1 Introduction

The construction of a compiler is one of the most comprehensive exercises in software engineering and formal language theory. A compiler does not merely translate text from one representation to another; it must rigorously verify that the input source code is well-formed at multiple levels of abstraction (lexical, syntactic, and semantic) before any meaningful code generation can take place. Each of these levels addresses a distinct class of errors and operates on a different representation of the program, forming a pipeline in which the output of one stage becomes the input of the next [1].

This document presents the design and implementation of a syntactic and semantic analyzer (Parser and SDT) built manually in Python, without the use of any parsing libraries, and integrated with the lexer from the previous project phase. The decision to implement every component from scratch — rather than relying on a parser-generator tool — makes every internal design decision explicit and traceable, and allows direct verification that the grammar satisfies the formal LL(1) conditions. [2] All internal structures produced during analysis are exported as PNG images so that every step of the process can be visually inspected.

- **Problem statement.** Implementing a compiler front-end requires not only tokenizing source code, but also verifying that the sequence of tokens conforms to a formal grammar and that the resulting constructs are semantically consistent. Raw token streams produced by a lexer carry no information about syntactic structure or semantic context, making it impossible to detect errors such as undeclared identifiers or type mismatches without a dedicated parser and semantic analysis pass. A program that is lexically valid — in the sense that every token belongs to a recognized category — may still be syntactically malformed (as in int x = ;) or semantically inconsistent (as in int x = y + 1; where y has not been previously declared). Detecting and reporting these two distinct classes of errors, with precise diagnostics and clear theoretical justification for each outcome, is the central problem this project addresses.

- **Motivation.** Understanding how parsers operate at the algorithmic level — constructing FIRST/FOLLOW sets, building LL(1) parsing tables, and traversing an explicit stack — provides a solid foundation for compiler theory. Implementing these components from scratch, rather than delegating to a parser-generator tool, makes every design decision visible and traceable. In particular, choices such as left factoring the grammar to eliminate non-determinism, resolving potential FIRST/FOLLOW conflicts, and defining the semantic rules that govern type propagation through arithmetic expressions must each be made explicitly and justified formally. This level of transparency is not achievable when using black-box tools, and it is precisely what allows the implementation to be understood, verified, and extended in future compiler phases.

- **Objective.** To implement a complete LL(1) top-down parser that reads tokens produced by the enriched lexer, builds a parse tree, performs Syntax-Directed

Translation (SDT) for basic semantic validation, and finally constructs an Abstract Syntax Tree (AST). The three tree structures are exported as PNG images to allow visual inspection of the analysis process. Beyond these concrete deliverables, the broader objective is to demonstrate that every component of the syntactic and semantic analysis pipeline — from FIRST/FOLLOW computation to symbol table management and type compatibility checking — can be implemented and understood from first principles, without abstraction layers that obscure the underlying theory.

2 Theoretical Framework

2.1 Compilers and Parsing

A compiler translates source code written in a high-level language into a lower-level representation that can be executed by a machine. The front-end of a compiler is responsible for three main stages: lexical analysis (tokenization), syntactic analysis (parsing), and semantic analysis. This project focuses on the latter two stages [**1**].

The lexical analysis stage scans the source code character by character and groups sequences of characters into tokens, each of which carries a category (such as keyword, identifier, operator, or punctuation), a lexeme, and a source position. The output of this stage is a flat token stream with no structural information. Syntactic analysis then takes this token stream and attempts to derive it from the productions of a formal context-free grammar (CFG), constructing a parse tree that makes the hierarchical structure of the program explicit. Finally, semantic analysis examines the constructs identified by the parser and verifies contextual constraints that cannot be expressed by a context-free grammar alone, such as type compatibility and declaration order.

A context-free grammar is formally defined as a 4-tuple $G = (V, T, P, S)$, where V is a finite set of non-terminal symbols, T is a finite set of terminal symbols disjoint from V, P is a finite set of productions of the form $A \to \alpha$ where $A \in V$ and $\alpha \in (V \cup T)^*$, and $S \in V$ is the designated start symbol. The language generated by G, denoted $\mathcal{L}(G)$, is the set of all strings of terminals derivable from S by applying productions in P. The parser's task is to determine whether the input token stream belongs to $\mathcal{L}(G)$ and, if so, to construct the corresponding derivation.

2.2 LL(1) Parsing

An LL(1) parser processes the input from **L**eft to right, constructs a **L**eftmost derivation, and requires only **1** token of lookahead to make each parsing decision without backtracking. The parser operates with an explicit stack initialized as `[$, S]`, where `$` denotes the end-of-input marker and `S` is the start symbol.

At each step, the parser inspects the top of the stack and the current lookahead token:

- If the top is a terminal matching the lookahead, the token is consumed and the stack is popped.

- If the top is a non-terminal, the LL(1) parsing table is consulted to determine which production to apply; the non-terminal is replaced by the right-hand side of the selected production, pushed onto the stack in reverse order so that the leftmost symbol ends up on top.

- If no entry exists in the table, a syntax error is reported.

The parser terminates successfully when both the stack and the remaining input reduce to the end-of-input marker `$`. This condition confirms that the entire input has been derived from the start symbol S using a leftmost derivation, which is the defining property of top-down parsing.

For a grammar to be parseable by an LL(1) parser, it must satisfy two structural requirements. First, it must be free of **left recursion**: a non-terminal A is left-recursive if $A \Rightarrow^* A\alpha$ for some string α, which would cause the parser to loop indefinitely when expanding A. Left recursion is eliminated by rewriting productions using right-recursive auxiliary non-terminals. Second, the grammar must be **left-factored**: if two productions for the same non-terminal share a common prefix, the parser cannot determine which to apply using only one token of lookahead. Left factoring isolates the common prefix into a single production and defers the decision to an auxiliary non-terminal, as was done in this project with `DECLARATION_PRIME`. [3]

2.3 FIRST and FOLLOW Sets

The LL(1) parsing table is constructed from two derived sets:

- **FIRST**(α): The set of terminals that can appear as the first symbol of any string derived from α. If α can derive the empty string ε, then ε is also included.

- **FOLLOW**(A): The set of terminals that can appear immediately to the right of non-terminal A in some sentential form. The end-of-input marker `$` is included if A can appear at the rightmost position.

A grammar is LL(1) if, for every non-terminal A and every pair of productions $A \to \alpha \mid \beta$, the sets $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$ are disjoint, and if $\varepsilon \in \text{FIRST}(\alpha)$ then $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$

The FIRST sets are computed by applying the following rules exhaustively:

1. If X is a terminal, $\text{FIRST}(X) = \{X\}$.

2. If $X \to \varepsilon$ is a production, add ε to $\text{FIRST}(X)$.

3. If $X \to Y_1 Y_2 \cdots Y_k$ is a production, add $\text{FIRST}(Y_1) \setminus \{\varepsilon\}$ to $\text{FIRST}(X)$. For each i from 1 to k, if $\varepsilon \in \text{FIRST}(Y_j)$ for all $j < i$, also add $\text{FIRST}(Y_i) \setminus \{\varepsilon\}$ to $\text{FIRST}(X)$. If $\varepsilon \in \text{FIRST}(Y_j)$ for all j, add ε to $\text{FIRST}(X)$.

The FOLLOW sets are computed by applying the following rules exhaustively:

1. Add \$ to FOLLOW(S), where S is the starts ymbol.
2. For each production $A \to \alpha B \beta$, add FIRST(β) $\setminus \{\varepsilon\}$ to FOLLOW(B).
3. For each production $A \to \alpha B$, or $A \to \alpha B \beta$ where $\varepsilon \in$ FIRST(β), add FOLLOW(A) to FOLLOW(B).

The LL(1) parsing table M is then populated as follows: for each production $A \to \alpha$, for each terminal $a \in$ FIRST(α), set $M[A, a] = A \to \alpha$. If $\varepsilon \in$ FIRST(α), then for each terminal $b \in$ FOLLOW(A), set $M[A, b] = A \to \alpha$. If any cell $M[A, a]$ receives more than one production, the grammar is not LL(1) and cannot be parsed deterministically with one token of lookahead. [6]

2.4 Syntax-Directed Translation (SDT)

Syntax-Directed Translation attaches semantic rules (attributes and actions) to grammar productions. After a successful parse, the SDT traverses the parse tree and evaluates these rules to perform semantic checks. There are two classes of attributes [4]:

- **Synthesized attributes:** The value of a synthesized attribute at a node is computed from the attribute values of its children. Synthesized attributes flow upward through the parse tree and are the natural mechanism for type inference in expressions: the type of an arithmetic expression is computed from the types of its operands.
- **Inherited attributes:** The value of an inherited attribute at a node is computed from the attribute values of its parent and siblings. Inherited attributes flow downward and are used, for example, to propagate the declared type of a variable into the expression on the right-hand side of an assignment for compatibility checking.

The semantic rules enforced in this project through SDT are:

- Building and maintaining a symbol table that records the declared type of each identifier.
- Detecting redeclaration of identifiers within the same scope.
- Verifying that identifiers are declared before use.
- Inferring and propagating types through arithmetic expressions using synthesized attributes.
- Validating type compatibility in assignments using the declared type as an inherited attribute passed to the expression.

The symbol table plays a central role in SDT: it is the data structure that bridges the gap between the declaration of an identifier and its subsequent use. Each entry in the symbol table stores at minimum the identifier's lexeme, its declared type, and the line at which it was declared. When an identifier is encountered in an expression, the symbol table is consulted to retrieve its type; if the identifier is absent, a semantic error is reported. This mechanism is what allows the SDT to enforce declaration-before-use as a semantic constraint that the context-free grammar alone cannot express.[7]

2.5 Abstract Syntax Tree (AST)

While the parse tree mirrors every production applied during parsing (including auxiliary non-terminals, epsilon nodes, and punctuation), the AST retains only the semantically relevant nodes. This compact representation is better suited for subsequent compiler phases such as intermediate code generation.

More precisely, the parse tree is a concrete representation of the derivation: every non-terminal introduced during parsing, including auxiliary symbols such as `DECLARATION_PRIME`, `EXPRESSION_PRIME`, and `TERM_PRIME` that exist to eliminate left recursion or enforce left factoring, appears as an explicit node. The parse tree is therefore coupled to the grammar's syntactic structure rather than to the semantic structure of the program.

The AST, by contrast, abstracts away these grammatical artifacts and represents only the logical structure of the program. Operator nodes directly connect to their operands, declaration nodes connect directly to their identifier and value, and punctuation tokens such as parentheses and semicolons are omitted entirely. This representation is language-semantics-driven rather than grammar-driven, and it is the standard input to the intermediate code generation phase of a compiler. [5]

In this implementation, three tree structures are produced and exported: the raw parse tree, which records every production applied; the verified parse tree, which annotates each node with its semantic state (*valid*, *error*, or *unevaluated*); and the AST, which retains only the nodes that carry semantic content. The visual distinction between these three structures makes the separation between syntactic and semantic analysis concrete and inspectable.

3 Development

3.1 System Pipeline

The complete analysis pipeline performs the following steps in order:

1. Reads the source input from `input.txt` (or from a file specified as a command-line argument).
2. Tokenizes the input using the enriched lexer.
3. Applies an LL(1) top-down parser with an explicit parsing table and an explicit stack.

4. Builds the **parse tree** from the sequence of productions applied.
5. Executes the **SDT** for basic semantic validation.
6. Builds the **verified parse tree** (an annotated copy of the parse tree).
7. Builds the **Abstract Syntax Tree (AST)**.
8. Exports all three trees as `.png` images to the `outputs/` directory.

3.2 Project Files

- `lexer.py` — Reusable lexer that produces detailed tokens.
- `parser_ll1.py` — Grammar definition, FIRST/FOLLOW computation, LL(1) parsing table, stack-based parser, parse tree construction, SDT, and AST construction.
- `tree_renderer.py` — Custom tree renderer that produces PNG images using the Pillow library.
- `main.py` — Entry point that orchestrates the full pipeline.
- `input.txt` — Default source input.
- `outputs/` — Directory where all generated artifacts are written.

3.3 Enriched Lexer

The original lexer printed only a flat sequence of token class names, for example:

```
keyword identifier operator constant punctuation
Total of tokens: 5
```

This was insufficient for the parser and the SDT because the following information needed to be preserved for each token: the lexeme itself, the exact token type, the source line and column, and the terminal symbol consumed by the grammar. The lexer was therefore extended to produce structured tokens with the form:

```
Token(kind, lexeme, line, column)
```

For example, the declaration `int result = 3;` produces:

```
Token("KEYWORD",    "int",    1,  1)
Token("IDENTIFIER", "result", 1,  5)
Token("OPERATOR",   "=",      1, 12)
Token("CONSTANT",   "3",      1, 14)
Token("PUNCTUATION",";",      1, 15)
```

A compatibility output is also retained so that running `lexer.py` directly still displays the legacy class-name sequence.

3.4 Extended Grammar for Arithmetic Expressions

To support simple arithmetic expressions, the grammar was formally extended to a full LL(1) grammar with operator precedence:

$$
\begin{aligned}
S &\rightarrow \text{DECLARATION} \\
\text{DECLARATION} &\rightarrow \text{TYPE identifier DECLARATION_PRIME} \\
\text{DECLARATION_PRIME} &\rightarrow = \ \text{EXPRESSION} \ ; \\
&\mid \ ; \\
\text{TYPE} &\rightarrow \texttt{int} \mid \texttt{float} \mid \texttt{char} \mid \texttt{double} \\
\text{EXPRESSION} &\rightarrow \text{TERM EXPRESSION_PRIME} \\
\text{EXPRESSION_PRIME} &\rightarrow + \ \text{TERM EXPRESSION_PRIME} \\
&\mid \ - \ \text{TERM EXPRESSION_PRIME} \\
&\mid \ \varepsilon \\
\text{TERM} &\rightarrow \text{FACTOR TERM_PRIME} \\
\text{TERM_PRIME} &\rightarrow * \ \text{FACTOR TERM_PRIME} \\
&\mid \ / \ \text{FACTOR TERM_PRIME} \\
&\mid \ \varepsilon \\
\text{FACTOR} &\rightarrow \text{identifier} \mid \text{constant} \mid (\ \text{EXPRESSION}\ )
\end{aligned}
$$

The terminal symbols recognized by the parser are: `int`, `float`, `char`, `double`, `identifier`, `constant`, `=`, `;`, `+`, `-`, `*`, `/`, `(`, `)`, `$`.

3.5 LL(1) Parser Implementation

The parser is implemented entirely by hand using the following components:

- **Parsing table** — Indexed by (non-terminal, terminal) pairs; each entry stores the production to apply.
- **Explicit stack** — Initialized as `[$, S]`; non-terminals are expanded and terminals are matched and consumed.
- **Step-by-step trace** — Each parsing step is recorded with the current stack contents, the remaining input, and the action taken.

The internal workflow is:

1. Build the grammar from the production rules.
2. Compute FIRST sets for all symbols.
3. Compute FOLLOW sets for all non-terminals.
4. Generate the LL(1) parsing table.

5. Parse the input starting from `[$, S]`; each expansion creates child nodes in the parse tree.

The following output files are written to `outputs/`: `first_follow.txt`, `parsing_table.txt`, and `parse_trace.txt`. The trace file records the step number, stack contents, remaining input, and applied action at every step.

3.6 Semantic Rules (SDT)

The SDT is executed after a successful parse. The following semantic rules are enforced:

- Declaration of an identifier with its declared type is recorded in the symbol table.
- Redeclaration of an already-declared identifier is flagged as an error.
- Use of an identifier that has not been previously declared is flagged as an error.
- The type of constant literals is inferred from their lexeme.
- Types are propagated through arithmetic sub-expressions using the following rules:
  - `int op int` → `int`
  - Any operand of type `float` promotes the result to `float`
  - Any operand of type `double` promotes the result to `double`
  - `char` does not participate in arithmetic operations
- Assignment compatibility is validated according to the table below:

Declared type	Compatible expression types
`char`	`char`
`int`	`int`
`float`	`int`, `float`
`double`	`int`, `float`, `double`

Table 1: Assignment type compatibility rules

4 Results

4.1 Test 1: Valid Declaration

Input: `int a = 5;`

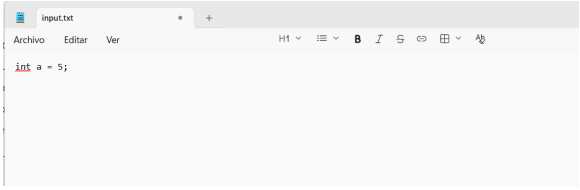

Figure 1: Input from Test 1

Result: Parse successful; SDT verified. Three tree images were generated: `parse_tree.png`, `verified_parse_tree.png`, and `abstract_tree.png`.

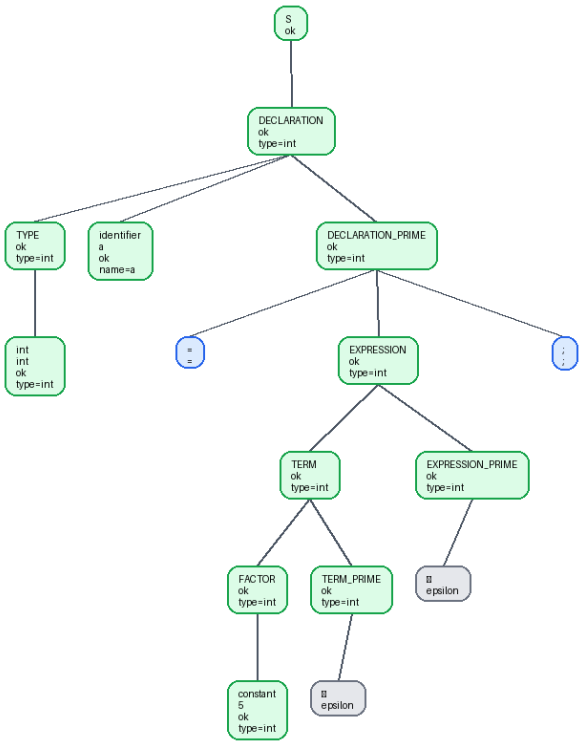

Figure 2: Verified parse tree from Test 1

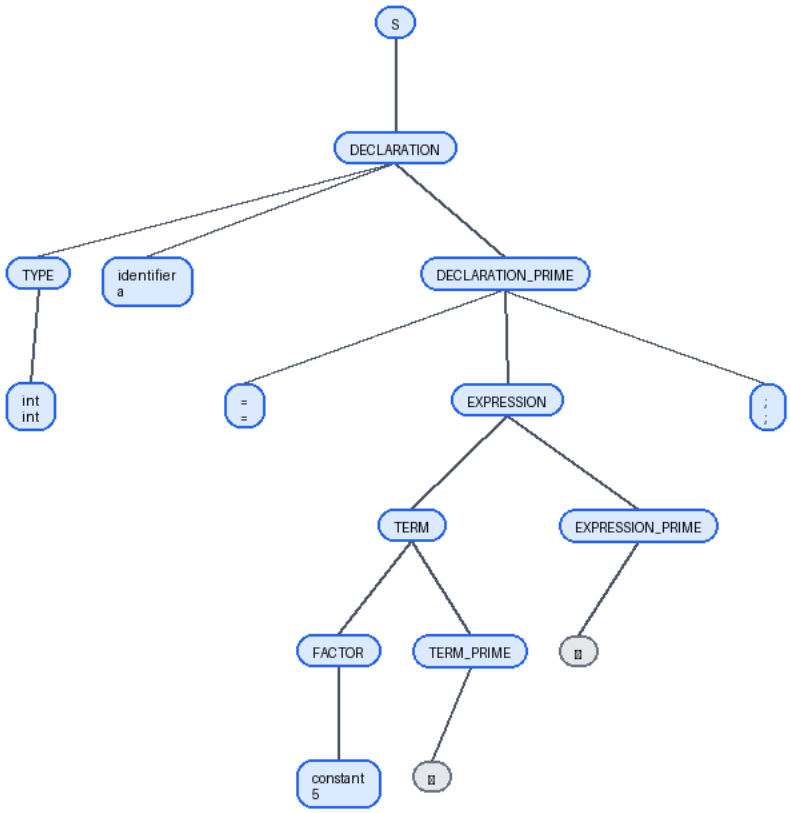

Figure 3: Parse tree

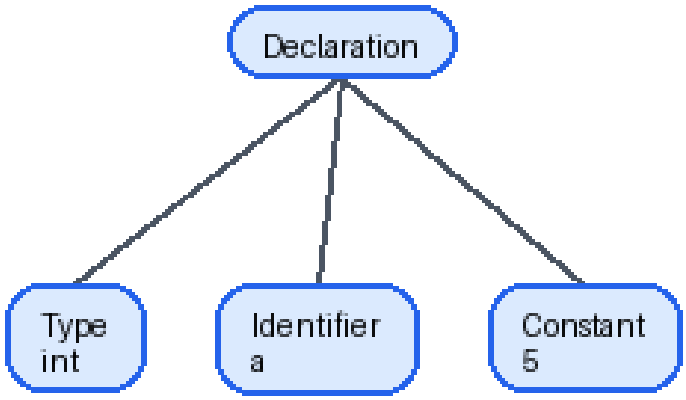

Figure 4: Abstract tree

4.2 Test 2: Valid Arithmetic Expression

Input: `int result = (3 + 5) * 2;`
Result:

```
Parsing Success!
SDT Verified!
```

The expression `(3 + 5) * 2` is correctly parsed using the extended arithmetic grammar. Type inference yields `int`, which is compatible with the declared type.

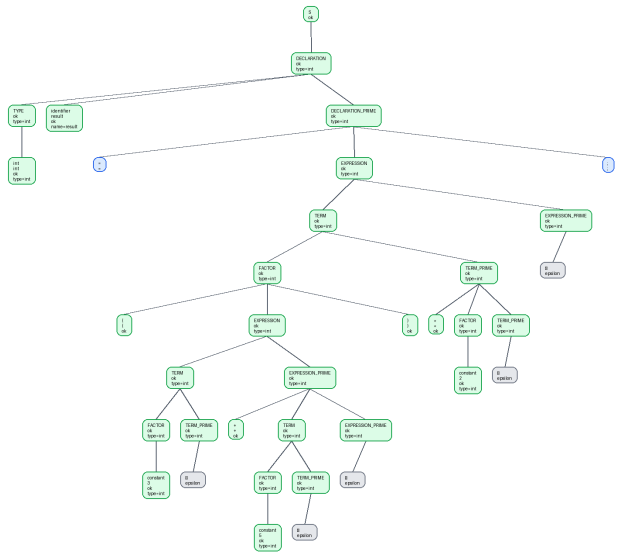

Figure 5: Verified parse tree from Test 2

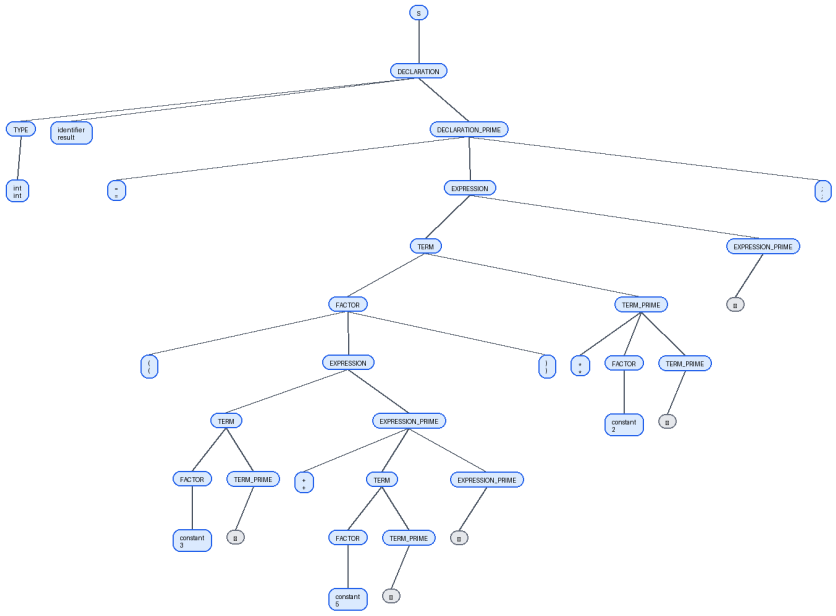

Figure 6: Parse tree

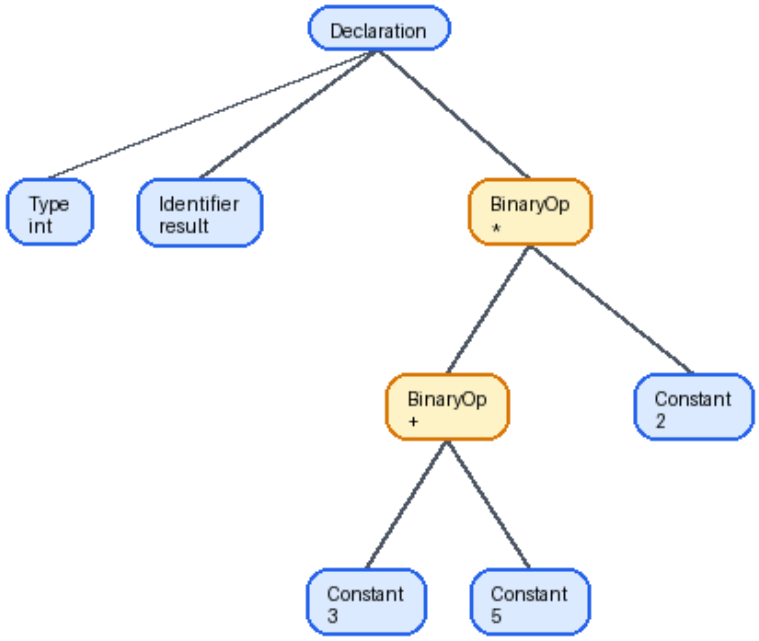

Figure 7: Abstract tree

4.3 Test 3: Semantic Error

Input: `int x = y + 1;`

Result: Parse succeeds; SDT fails because identifier y was used without a prior declaration. The verified parse tree marks the corresponding node in red and records the semantic error message.

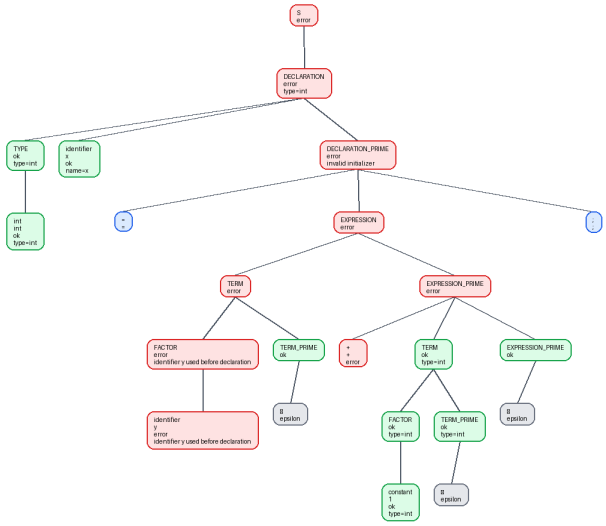

Figure 8: Verified parse tree from Test 3

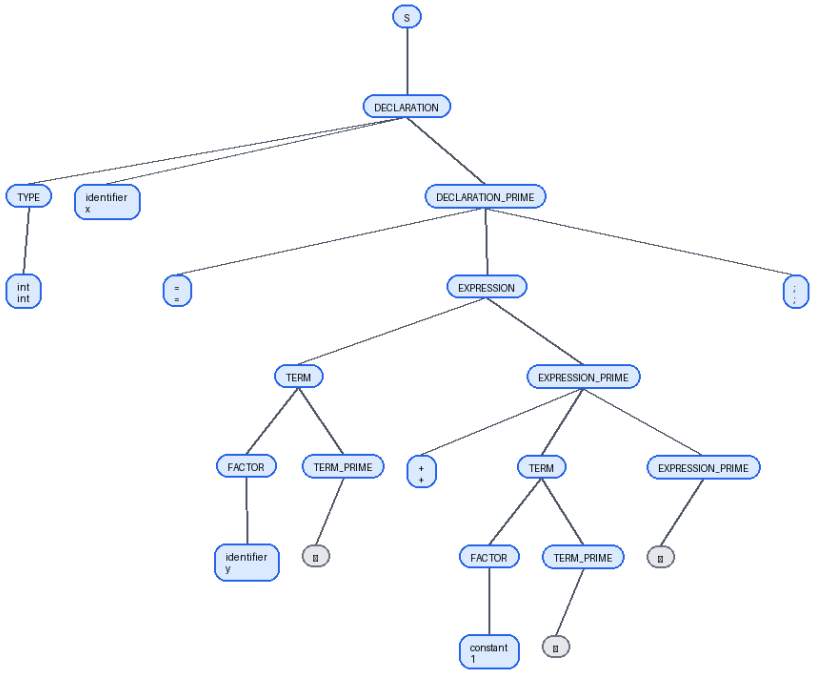

Figure 9: Parse tree

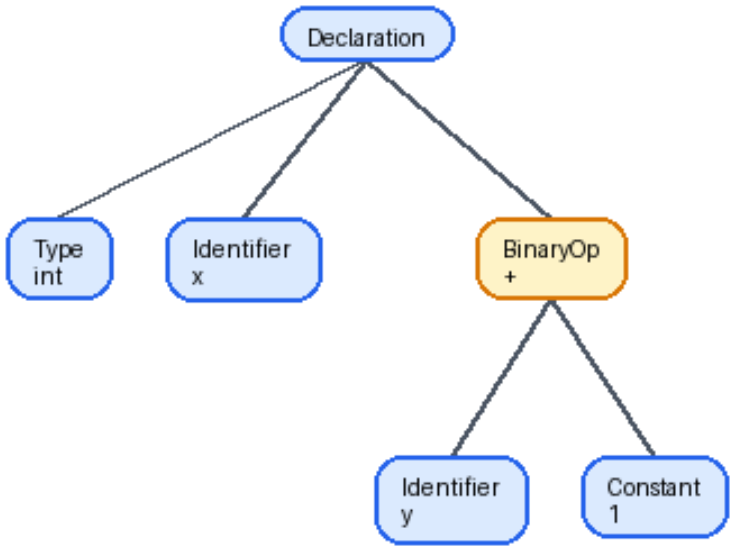

Figure 10: Abstract tree

4.4 Test 4: Syntax Error

Input: `int x = ;`

Result: The parser fails when it encounters `;` where an expression is expected. The SDT is not executed. The parse trace records the exact step at which the error was detected.

```
Parsing error...
No production for EXPRESSION with lookahead ';' (';' at line 1, column 9).
```

Figure 11: Error from Test 4

4.5 Generated Output Files

Each successful execution writes the following files to `outputs/`:

File	Description
`first_follow.txt`	FIRST and FOLLOW sets for all grammar symbols
`parsing_table.txt`	Full LL(1) parsing table
`parse_trace.txt`	Step-by-step parsing trace
`parse_tree.png`	Complete parse tree with all productions
`verified_parse_tree.png`	Annotated parse tree with semantic state per node
`abstract_tree.png`	Compact AST with only semantically relevant nodes

Table 2: Output artifacts generated by the pipeline

The visual conventions used in the verified parse tree are: **green** for valid nodes, **red** for semantic errors, **grey** for epsilon nodes, **blue** for structural nodes without a semantic annotation, and **yellow** for binary operator nodes in the AST.

5 Conclusions

The implementation of a manual LL(1) parser and SDT in Python demonstrated that building each compiler front-end component from first principles provides a clear and practical understanding of syntactic and semantic analysis. By explicitly computing FIRST and FOLLOW sets, constructing the parsing table, performing stack-based parsing, and evaluating semantic rules, the project made visible the full decision process followed by a predictive parser.

The resulting system successfully integrates lexical input, syntactic validation, semantic verification, and structural representation through the generation of a parse tree, a verified parse tree, and an Abstract Syntax Tree (AST). The project satisfies the objective of distinguishing among syntactically and semantically correct inputs, syntactically correct but semantically invalid inputs, and syntactically invalid inputs.

Beyond its functional outcome, the project also reinforced the theoretical relationship between formal grammars, LL(1) parsing, and Syntax-Directed Translation. Implementing these mechanisms manually, rather than relying on external parsing tools, helped clarify how compiler front-end stages interact and why semantic analysis must be treated as a distinct phase from syntax checking.

The project have limitations, among this are support for only a single declaration per execution and the absence of relational operators, boolean expressions, nested scopes, and function definitions. These limitations can be work, such as extending the grammar, enriching the semantic rules, and incorporating additional language constructs to have a more complete compiler.

6 References

[1] IBM, "What is a compiler?," *IBM Think*, [Online].
Available: `https://www.ibm.com/mx-es/think/topics/compiler`. [Accessed: Mar. 2026].

[2] Wikipedia, "LL parser," *Wikipedia, The Free Encyclopedia*, [Online].
Available: `https://en.wikipedia.org/wiki/LL_parser`. [Accessed: Apr. 2026].

[3] GeeksforGeeks, "Construction of LL(1) Parsing Table," *GeeksforGeeks*, [Online].
Available: `https://www.geeksforgeeks.org/construction-of-ll1-parsing-table/`. [Accessed: Apr. 2026].

[4] GeeksforGeeks, "Syntax directed translation in compiler design," *GeeksforGeeks*, [Online].
Available: `https://www.geeksforgeeks.org/syntax-directed-translation-in-compiler-design/`. [Accessed: Apr. 2026].

[5] Wikipedia, "Abstract syntax tree," *Wikipedia, The Free Encyclopedia*, [Online]. Available: `https://en.wikipedia.org/wiki/Abstract_syntax_tree`. [Accessed: Apr. 2026].

[6] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Upper Saddle River, NJ, USA: Pearson Addison-Wesley, 2006.

[7] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed. Burlington, MA, USA: Morgan Kaufmann, 2011.